Clic Justo · Plataforma de Datos y Marco de QA

Un sistema, tres capas de pruebas y un pipeline que avisa cuando algo se rompe

Integrantes del Equipo

- **Jonatan Aguilar** (Carné: 20240565)
  josue.aguilar@ufm.edu
- **Hugo Galina** (Carné: 20240566)

Información Institucional

- **Curso:** Calidad y Automatización de Software
- **Universidad:** Universidad Francisco Marroquín
- **Ciclo:** Segundo Semestre, Noviembre 2026

1. El Problema y Contexto de Negocio

Modelo de Negocio Publicitario

- Plataforma **Cost-Per-Click (CPC)** que conecta anunciantes locales con medios digitales (prensa, blogs, portales).
- Los anunciantes definen presupuestos diarios estrictos.
- Los medios reciben transferencias proporcionales al tráfico legítimo generado.
- **Reto crítico:** Detección de clics fraudulentos (bots, emuladores y tráfico inválido) antes de consolidar el cobro.

Riesgos de Calidad Mitigados

- **Sobrecobro indebido:** Agotamiento artificial del presupuesto de pequeños comerciantes.
- **Inconsistencia contable:** Descuadres financieros entre clics brutos e ingresos facturados.
- **Falsos positivos operacionales:** Pausar injustificadamente campañas comerciales legítimas por baselines erróneos.

2. Arquitectura Lambda del Sistema

Flujo de Datos y Capas

- **Speed Layer:** Ingesta incremental asíncrona, validación de contratos JSON y evaluación de semáforos sub-minuto.
- **Batch Layer (Medallion):**
  - **Bronze:** Eventos crudos inmutables.
  - **Silver:** Deduplicación y sanitización de datos sensibles (PII).
  - **Gold:** Baselines históricos horarios y agregaciones en **DuckDB**.
- **Serving Layer:** API REST construida sobre **FastAPI** con latencia sub-segundo.
- **Interfaz Web:** 3 pantallas operativas para monitoreo visual y control.

Pila Tecnológica

- **Lenguaje:** Python 3.14 (gestión determinista con `uv`).
- **Motor Analítico:** DuckDB + almacenamiento columnar Parquet.
- **Serving API & UI:** FastAPI + HTML5 semántico y CSS embebido.
- **Automatización de QA:** Pytest, Playwright (POM), Ruff y MyPy.
- **Integración Continua:** GitHub Actions con alertas SMTP reales.

3. Estrategia de Calidad: Pirámide de Mike Cohn

Distribución de las 48 Pruebas Automatizadas

Capa de la Pirámide	Casos	Tiempo	Enfoque
Capa 3: E2E Playwright	3	1.71s	Flujos de usuario
Capa 2: Integración HTTP	10	0.27s	Caja negra / BVA
Capa 1: Unitarias & DDT	35	0.45s	Lógica / Contratos
TOTAL GENERAL	**48**	**2.43s**	**61.58% Cobertura**

Principio Shift-Left Testing

- Detección temprana de anomalías antes de alcanzar la base de datos o producción.
- Linter **Ruff** y tipado estricto **MyPy** ejecutados como etapa 1.
- Validación de esquemas en ingesta: eventos malformados se desvían de inmediato a **cuarentena**.

4. Capa 1: Pruebas Unitarias y Dirigidas por Datos (DDT)

Contratos JSON Schema Multiversión

- Validación exhaustiva de contratos bajo **JSON Schema draft 2020-12**:
  - `test_contratos_v1.py` (12 pruebas): Verificación campo por campo de tipos y restricciones en eventos de clic e impresión.
  - `test_contratos_multiversion.py` (2 pruebas): Evolución de esquema y compatibilidad hacia atrás.
- Detección de fallos en fracciones de segundo con aislamiento absoluto.

Lógica de Negocio y Reglas Gold

- `test_cuarentena_empaquetado.py` (2 pruebas): Enrutamiento hacia crudo o zona de cuarentena.
- `test_reglas_negocio_gold.py` (4 pruebas): Evaluación de acciones contractuales (`normal`, `revisar`, `pausar`).
- `test_privacidad_pii_y_reglas.py` (5 pruebas DDT): Matriz de decisión ante presupuestos agotados y picos de tráfico.
- `test_transformaciones_sql.py` (2 pruebas): Integridad transaccional de DuckDB.

5. Capa 2: Integración HTTP y Caja Negra (ISO/IEC/IEEE 29119)

Partición de Equivalencia (Equivalence Partitioning)

- **Clase Válida:** Rutas operativas y de interfaz entregando HTTP 200 (`/health`, `/`, `/ui/`, `/ui/campanias`, `/ui/analitica`).
- **Clase Inválida (No Encontrado):** Consulta de ID inexistente (`999999`) retorna HTTP 404/503.
- **Clase Inválida (Tipo de Dato):** Consulta con cadena alfanumérica (`invalido_str`) activa la validación de Pydantic retornando **HTTP 422 Unprocessable Entity**.

Análisis de Valores Límite (BVA) y SLA

- **BVA sobre `campania_id`:**
  - `-1`: Límite inferior fuera de dominio.
  - `0`: Frontera entre enteros negativos y positivos.
  - `1`: Primer identificador entero positivo válido.
- **Benchmark de SLA (ADR-001):** `test_benchmarks_adrs.py` valida que la ingesta asíncrona mantenga un rendimiento sub-segundo bajo carga.

6. Capa 3: Pruebas End-to-End con Playwright

3 Pantallas Operativas en FastAPI

- **Dashboard (`/ui/`):** Estado de salud en tiempo real (`#badge-health`), indicadores de ingesta y metadatos de despliegue.
- **Campañas y Semáforo (`/ui/campanias`):** Input `#input-campania-id`, botón `#btn-consultar`, presupuesto y distintivo semafórico (`#badge-accion`).
- **Explorador Analítico (`/ui/analitica`):** Selector interactivo `#select-query` (P1, P2, P3), botón `#btn-ejecutar` y tabla `#tabla-resultados`.

Suite de Flujos de Usuario

- **Flujo 1 (Smoke Test - `@pytest.mark.smoke`):** Validación de carga del dashboard, estado operacional y enlaces de navegación en 1.1s.
- **Flujo 2 (Inspección y Semáforo):** Búsqueda de campaña estándar 8596 (`normal`) y campaña con presupuesto agotado 1001 (`pausar`).
- **Flujo 3 (Explorador Analítico):** Ejecución de consultas P1 y P2 verificando el renderizado tabular de 5 filas.

7. Principios SOLID en el Page Object Model (POM)

SRP: Single Responsibility Principle

- **`BasePage`:** Responsabilidad única de interactuar con el navegador Playwright (navegación, esperas explícitas de visibilidad y resolución de URLs). Cero conocimiento de selectores de negocio.
- **Page Objects Concretos:** `DashboardPage`, `CampaignPage` y `AnalyticsPage` encapsulan única y exclusivamente los selectores y acciones semánticas de su pantalla.

DIP: Dependency Inversion Principle

- En `base_page.py` se formalizó el protocolo abstracto:

```
@runtime_checkable
class IPage(Protocol):
    def navegar(self, ruta_relativa: str) -> Response |
None: ...
    def obtener_titulo(self) -> str: ...
    def obtener_texto(self, selector: str) -> str: ...
```

- Las pruebas dependen de la abstracción contractual `IPage`, desacoplándose de Playwright.

8. Puerta de Calidad (Quality Gate) y Cobertura

Justificación del Umbral del 60%

- Umbral fijado en el pipeline: `--cov-fail-under=60`.
- Cobertura global alcanzada: **61.58% (484/786 sentencias)**.
- **Módulos críticos con alta cobertura (80% - 100%):**
  - `contratos.py` (82%), `enrutador.py` (100%), `config.py` (100%), `modelos.py` (100%), `rutas_ui.py` (100%), `motor.py` (96%).
- **Módulos excluidos justificadamente:**
  - `ejecutar_pipeline.py` (0%): Script wrapper CLI para ingesta manual de terabytes en disco local, inviable en runners de CI.

Comportamiento del Pipeline

- Si un commit disminuye la cobertura por debajo del 60%:

```
FAIL Required test coverage of 60% not reached.
Total coverage: 58.20%
```

- El pipeline aborta en la Etapa 2 impidiendo la construcción del artefacto y el despliegue.

9. Pipeline de CI/CD (GitHub Actions)

Secuencia estricta de 5 etapas protegidas por compuertas de calidad:

1. Lint & Tipos (Ruff & MyPy) $\rightarrow$ **2. Unitarias & Cobertura** ($\geq 60\%$) $\rightarrow$ **3. Build Artefacto** (uv.lock) $\rightarrow$ **4. Despliegue & Integración** (HTTP) $\rightarrow$ **5. End-to-End** (Playwright)

Rendimiento y Determinismo

- Tiempo total de ejecución: **25 a 35 segundos** (meta de la rúbrica: < 3 minutos).
- Gestión de paquetes determinista con `uv sync --frozen`.
- Timeouts de 5 minutos por etapa para prevenir procesos colgados.

Verificación de Identidad de Despliegue

- El pipeline levanta la instancia e invoca `/health`.
- Valida que la respuesta contenga el `COMMIT_SHA` exacto y el `APP_ENV` correspondiente.

10. Multiambiente y Parámetros Configurables

Dos Ambientes Aislados

- **Ambiente `staging`:** Desplegado en el puerto **8000**.
- **Ambiente `produccion`:** Desplegado en el puerto **8001**.
- Misma imagen y artefacto determinista, diferenciados por configuración inyectada por entorno.
- La instancia responde su identidad por HTTP, no por variable estática del CI.

Parámetros Configurables (`workflow_dispatch`)

1. **`ambiente`:** `staging` (default) o `produccion`.
2. **`test_suite`:** `smoke` (ejecución rápida) o `completa` (regresión integral).
3. **`browser`:** `chromium` (default) o `firefox`.
4. **`destinatario_correo`:** Correo dinámico para alertas (default: `josue.aguilar@ufm.edu`).

11. Notificaciones de Alerta en Tiempo Real (SMTP)

Arquitectura de Notificación

- Despacho automático ante fallo en **cualquiera de las 5 etapas** utilizando la acción `dawidd6/action-send-mail`.
- Conexión SMTP real autenticada (servidor `smtp.gmail.com`, puerto `465` con SSL).
- Cero simuladores locales: entrega real comprobada en bandejas de entrada.

Seguridad y Contenido de la Alerta

- **Seguridad:** Credenciales inyectadas desde **GitHub Secrets** (`SMTP_USERNAME`, `SMTP_PASSWORD`), enmascaradas en los logs públicos.
- **Contenido del Correo:**
  - Número de build y commit SHA fallido.
  - Etapa exacta que abortó la ejecución.
  - Ambiente probado y enlace directo al log de error.

12. Análisis Crítico de Sugerencias de Asistentes de IA

Sugerencia de la IA	Decisión	Justificación Técnica y Evidencia
1. Usar Node.js, React y Tailwind para la interfaz	**RECHAZADA**	Duplicaba la complejidad del pipeline e incrementaba el build a más de 3 minutos por descarga de paquetes NPM. Se implementó HTML5 nativo servido por FastAPI (< 20ms).
2. Mock total de HTTP y DuckDB en E2E	**RECHAZADA**	Desvirtuaba la Capa 3 de Cohn convirtiendo pruebas de sistema en unitarias. Se diseñó un fixture que arranca una instancia real de FastAPI con Uvicorn en background.
3. Protocolo `IPage` (DIP) y esperas explícitas en `BasePage`	**ACEPTADA**	Eliminó el flakiness y las condiciones de carrera, garantizando desacoplamiento arquitectónico y cumplimiento de principios SOLID.

13. Conclusiones y Demostración en Vivo

Cumplimiento Total de la Rúbrica

- 3 capas de pruebas completas según la Pirámide de Mike Cohn (48 pruebas pasando).
- Puerta de cobertura estricta (61.58% $\geq 60\%$).
- POM con principios SOLID (SRP y DIP) e interfaz web funcional.
- Pipeline multiambiente en < 35 segundos con alertas SMTP.
- Autoría acreditada en el historial de Git para cada integrante.

Guion de la Demostración (15 Minutos)

1. **Build Verde:** Demostración de las 5 etapas ejecutadas.
2. **Parámetros:** Lanzamiento en ambiente `produccion` y suite `smoke`.
3. **Fallo Inducido:** Modificación en el código del sistema según la capa solicitada.
4. **Alerta SMTP:** Verificación de la recepción del correo en vivo.
5. **Sesión de Preguntas.**

Muchas gracias por su atención. Espacio abierto para preguntas.